

## DIVIDE AND CONQUER

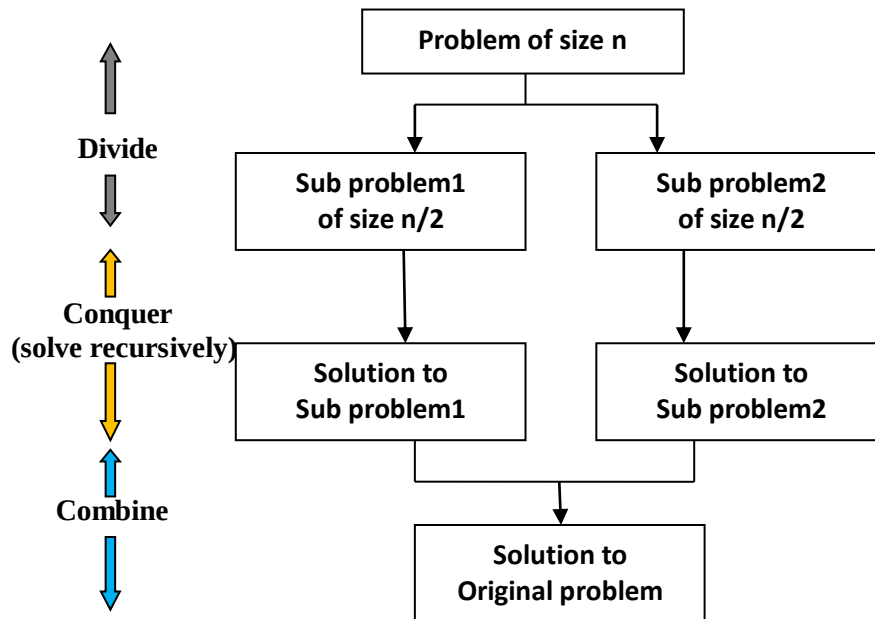
### GENERAL METHOD OF DAC:

- It is an one of the algorithmic strategy.
- The general method of Divide and Conquer is a problem-solving technique that solves a large problem by breaking it into smaller sub problems, solving them independently, and combining their solutions.

### GENERAL STEPS:

1. **Divide**
  - Break the original problem into two or more smaller sub problems of the same type.
2. **Conquer**
  - Solve the smaller sub problems recursively.
  - If a sub problem is small enough (base case), solve it directly.
3. **Combine**
  - Merge the solutions of the sub problems to obtain the solution to the original problem.

**Divide and Conquer technique is as shown in the form of figure:**



**Fig: Divide and Conquer technique**

### CONTROL ABSTRACTION OF DAC:

The control abstraction describes the general recursive structure used by a Divide and Conquer algorithm.

1. Given a problem 'P' of size  $n=2^K$ .
2. Recursive Algorithm for DAC(P):
  - If 'n' is so small, solve it directly.
  - If 'n' is large then divide 'P' into two sub problems p1 & p2. Then size is  $n/2=2^{K-1}$ .
  - DAC(P1) :solve each sub problem recursively.

- DAC(P2)
- Combine solutions of sub problems p1 & p2.

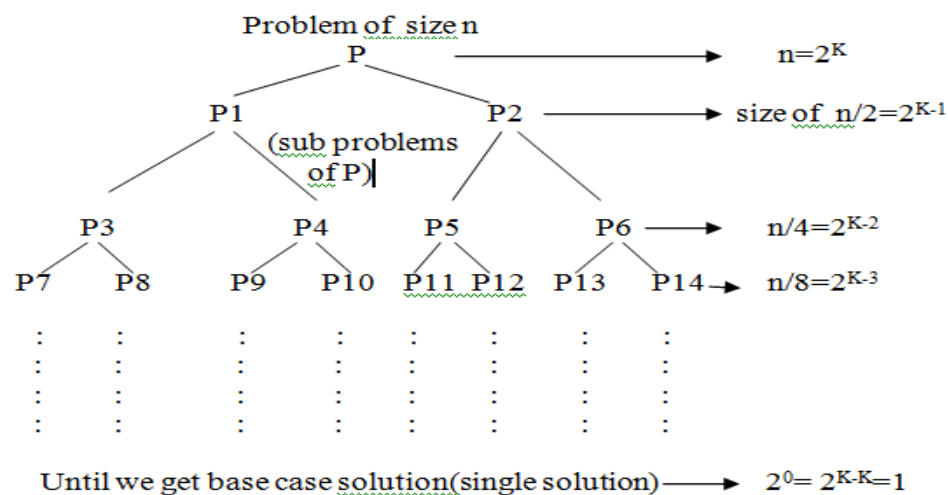
### **Algorithm for DAC:**

Algorithm DAC(P)

```
{
  if(small(P)) then
    return S(P)          //return solution of p
  else
  {
    //if large
    K=Divide(P) //obtain p1,p2,.....pK sub problems
    Apply DAC(P1), DAC(P2),..... //Recursively solves each sub problem.
    //combine the solutions of sub problems to obtain final solution
    return combine(DAC(P1),DAC(P2),.....DAC(PK))
  }
}
```

### **Eg: Recursive call tree for above algorithm**

For the Divide and Conquer control abstraction, the recursive-call tree can be represented like this:



The Height of the tree is reduced from  $2^K$  to  $2^{K-1}$ ..... to  $2^0$ .

At last level the problem size becomes 1:

$$2^{k-i}=2^0$$

$$k-i=0$$

$$i=k$$

$$\text{since } k = \log_2 n$$

Therefore Height of recursion tree =  $\log_2 n$

## General Recurrence Relation :

For a problem divided into **k** sub problems:

$$T(n) = aT(n/b) + f(n)$$

Where:

- **a** = number of sub problems
- **n/b** = size of each sub problem
- **f(n)** = time required for dividing and combining
- **T(n)** = total time complexity

## Calculate Divide and Conquer Recurrence Relation

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{if } n \text{ is large} \end{cases}$$

### 1. Base Case (Small Problem)

$$T(n) = g(n)$$

When the input size **n** is very small, we solve the problem directly.

**Example:**

If  $n=1$ , no division is required.

$$T(1) = 1$$

Here:

- $g(n)$  = time required to solve a small problem.

### 2. Recursive Case (Large Problem)

$$T(n) = T(n_1) + T(n_2) + \dots + T(n_k) + f(n)$$


Time to solve sub problems

- If **n** is small, then we will get  $g(n)$  constant time
- If **n** is large then we will get two comparisons of size  $(n/2)$  &  $(n/2)$ , Therefore it defined in terms of  $T(n/2)$  and  $T(n/2)$
- $f(n) = O(n) \rightarrow$  merging takes linear time

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small} \\ T(n/2) + T(n/2) + f(n) & \text{if } n \text{ is large} \end{cases}$$

It can be written as

$$T(n) = \begin{cases} 1 & \text{if } n \text{ is small} \\ 2T(n/2) + f(n) & \text{if } n \text{ is large} \end{cases}$$

**Standard recurrence equation for DAC:**

$$T(n) = 2T(n/2) + n$$

### Efficiency Analysis Of DAC By Using Master Theorem And Backward Substitution:

Here calculate time complexity by using recurrence methods.

**General recurrence relation equation is:**

$$T(n)=aT(n/b)+f(n)$$

**Standard recurrence equation for DAC:**

$$T(n)=2T(n/2)+n$$

**i)Using Master's Theorem:**

DAC Recurrence equation

$$T(n)=2T(n/2)+n$$

**Here  $a=2$  ,  $b=2$ ,  $f(n)=n$**

**Case1:**

$$a < b^d \quad d=1$$

$$2 < 2^1$$

$$2 < 2 \text{ [False]}$$

**Case 2:**

$$2=2 \text{ [True]} \rightarrow \text{case 2 valid for defining time complexity.}$$

**Case 3:**

$$a > b^d \quad d=1$$

$$2 > 2^1$$

$$2 > 2 \text{ [false]}$$

**Therefore apply time complexity for case 2 :**

$$T(n) = \Theta(n^d \log n)$$

$$= \Theta(n^1 \log n) = \Theta(n \log n)$$

**Hence Time complexity for DAC is:**

$$T(n) = \Theta(n \log n) \text{ or } O(n \log n)$$

**ii) Using Backward substitution:**

Consider DAC equation:

$$T(n)=2T(n/2)+n$$

Step 1:

$$T(n)=2T(n/2)+n \text{ -----eq(1)}$$

Let  $n=n/2$ , we get

$$T(n/2)=2T(n/4)+n/2 \text{ -----eq(2)}$$

Substitute eq(2) in (1) we get

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + n + n$$

$$T(n) = 4T(n/4) + 2n \text{ -----eq(3)}$$

Let  $n=n/4$ , we get

$$T(n/4) = 2T(n/8) + n/4 \text{ -----eq(4)}$$

Substitute eq(4) in (3) we get

$$T(n) = 4(2T(n/8) + n/4) + 2n$$

$$T(n) = 8T(n/8) + n + 2n$$

$$T(n) = 8T(n/8) + 3n$$

:  
:  
:

After k substitutions

$$T(n) = 2^k T(n/2^k) + 3n$$

Assume  $2^k = n$

$$K = \log n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$T(n) = nT(n/n) + Kn$$

$$= n(T(1)) + n \log n \quad \{T(1)=1\}$$

$$T(n) = n + n \log n$$

**Hence Time complexity for DAC is:**

$$T(n) = \Theta(n \log n) \text{ or } O(n \log n)$$